

Capabilities & API

1. Binocular Display

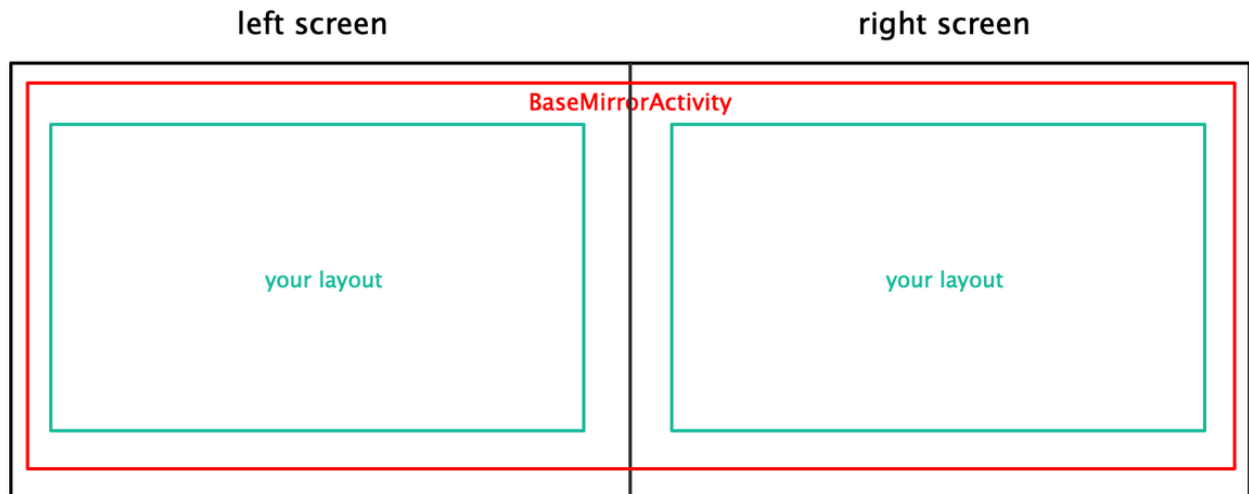
To implement Binocular Display, in addition to duplicating the same UI layout on both the left and right sides, any modification made to the left view must also be applied to the right layout. These requirements introduce a significant amount of boilerplate code. To address this issue, the SDK provides the `BindingPair` utility class based on `ViewBinding`. After passing the `ViewBinding` objects corresponding to the left and right layouts to its constructor, you can:

- Use the `updateView` method to operate on both layouts simultaneously.
- Use the `setLeft` method to operate only on the left layout.
- Use the `checkIsLeft` method within the `updateView` block to determine whether the current operation is on the left or right layout.

1.1 Binocular Display Components

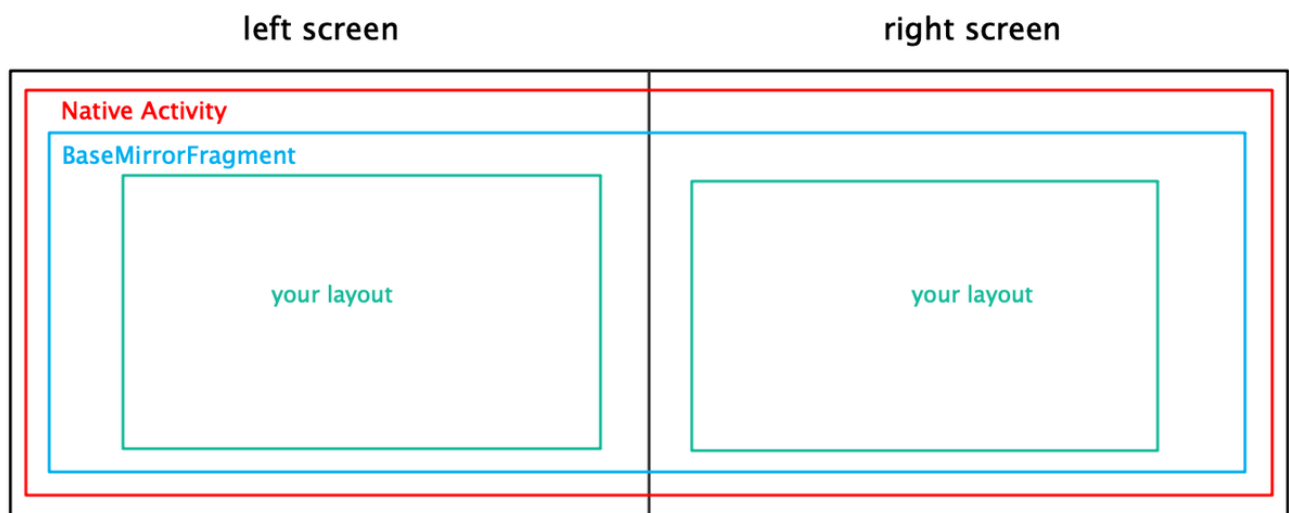
Binocular Display operations are primarily handled by the `BindingPair` utility class based on `ViewBinding`. The following UI components are encapsulated based on `BindingPair`:

- **Activity Level:** `BaseMirrorActivity`
 - Implements the logic for automatically mirroring the layout view of an Activity. Developers only need to focus on the business UI requirements and develop layout files in the native XML layout manner to achieve Binocular Display effects. Refer to `FusionVisionActivity` in the sample or any subclass of `BaseMirrorActivity` for implementation details.



- **Fragment Level:** `BaseMirrorFragment`

- Implements Binocular Display encapsulation at the Fragment level. It is added in the same way as native Fragments but should not be added to a `BaseMirrorActivity`. Refer to `FragmentDemoActivity` in the sample for implementation details.



- **View Level:**

- `MirrorContainerView` ** (Composition-based)**: Refer to `MirrorContainerViewActivity` for usage. Ensure that the `mirrorContainer.bindTo` method is implemented; otherwise, Binocular Display will not take effect.

- **BaseMirrorContainerView** ** (Inheritance-based)**: Refer to `com.ffalcon.mercury.android.sdk.demo.ui.wedget.TitleView` for usage. Note that `MirrorView` should not be added to a `BaseMirrorActivity`.
- **Toast:** `FToast`
 - A universal Toast encapsulation that supports Binocular Display. Refer to `FusionVisionHomeActivity` for usage.
- **Dialog:** `FDialog`
 - A universal Dialog encapsulation that supports Binocular Display. Refer to `DialogActivity` for usage.

Except for `FToast` and `FDialog`, which are independent components, other Binocular Display components can be reasonably assembled according to business requirements to achieve a perfect dual-screen Binocular Display effect.

1.2 `updateView` & `setLeft`

`mBindingPair.updateView{}`:

All Binocular Display components mentioned above use the `BindingPair` utility class to achieve left-right layout mapping. A single layout file is mapped to both the left and right screens, so there are actually two layouts in memory. To handle 繁琐的镜像操作, the SDK provides the `mBindingPair.updateView` method to simultaneously update the same component in both layouts.

Example:

Code block

```
1  // All operations within the updateView block will be mapped to both left and
   // right layouts
2  mBindingPair.updateView {
3      tvTitle.text = "my title"
4  }
5  // Equivalent to:
6  mBindingPair.left.tvTitle.text = "my title"
7  mBindingPair.right.tvTitle.text = "my title"
```

Note: Due to this feature, it is recommended to perform only UI-level update operations within the `updateView` block. Modifying external source data or binding events in this method will cause these operations to execute twice, leading to unpredictable consequences.

For external data updates or event binding, use the `mBindingPair.setLeft` method.

`mBindingPair.setLeft{}:`

Code block

```
1  mBindingPair.setLeft {
2      focusHolder.addFocusTarget(
3          // Typically, events are bound to the left view component
4          FocusInfo(
5              btnEvent,
6              eventHandler = { action -> handleAction(action) },
7              focusChangeHandler = { hasFocus ->
8                  // If you need to synchronously update the components on both
// sides in response to an event, you can call the updateView method again at
// this time.
9                  mBindingPair.updateView {
10                      triggerFocus(hasFocus, btnEvent,
11                          mBindingPair.checkIsLeft(this))
12                      }
13              )
14          }
15      focusHolder.currentFocus(mBindingPair.left.btnEvent)
16  }
```

Of course, if in complex business scenarios, you need to integrate logic in `updateView` but don't want it to be called twice, you can also judge the current execution area within the block of `updateView`.

Code block

```
1  mBindingPair.updateView {
2      // Do something...
3
4      // Check if it's the left layout
5      if (mBindingPair.checkIsLeft(this)){
6          // Only execute this code on the left layout
```

```
7     }  
8 }
```

Refer to the sample code for more specific scenario implementations.

2. Focus Management

Since the glasses screen cannot be directly touched like a phone, focus switching must be manually managed. Typically:

- Swiping forward and backward switches the focus.
- A single click triggers the event of the focused view.
- A double click exits the focus and returns to the previous level.

The simplest implementation of focus management involves maintaining an index where different indices correspond to different focusable views. However, this approach is not easily maintainable for complex pages.

A more structured approach:

Code block

```
1  // 1. Create a FocusHolder object  
2  val focusHolder = FocusHolder(true)  
3  ....  
4  
5  // 2. Use views from the left layout as focus targets  
6  mBindingPair.setLeft {  
7      // 2.1 Create a FocusInfo object  
8      val btn1Info = FocusInfo(  
9          btn1,  
10         eventHandler = { action ->  
11             // Handle events  
12             when (action) {  
13                 is TempleAction.Click -> {  
14                     FToast.show("bt1 click")  
15                 }  
16                 else -> Unit  
17             }  
18         },
```

```

19         focusChangeHandler = { hasFocus ->
20             // Handle focus changes
21             mBindingPair.updateView {
22                 btn1.setBackgroundColor(getColor(if(hasFocus)
R.color.purple_200 else R.color.black))
23             }
24         }
25     )
26
27     // 2.2 Add multiple FocusInfo objects for the same level
28     focusHolder.addFocusTarget(
29         // First focus position
30         btn1Info,
31         // Second focus position
32         FocusInfo(
33             btn2,
34             eventHandler = {action ->
35                 when(action) {
36                     is TempleAction.Click -> {
37                         FToast.show("bt2 click")
38                     }
39                     else -> Unit
40                 }
41             },
42             focusChangeHandler = { hasFocus ->
43                 mBindingPair.updateView {
44                     btn2.setBackgroundColor(getColor(if(hasFocus)
R.color.purple_200 else R.color.black))
45                 }
46             }
47         ),
48         // More focus positions...
49     )
50
51     // 2.3 Set the default focus position
52     focusHolder.currentFocus(mBindingPair.left.btn1)
53 }
54
55 // 3. Create a FixPosFocusTracker object
56 fixPosFocusTracker = FixPosFocusTracker(focusHolder).apply {
57     // 3.1 Set the initial focus state
58     focusObj.hasFocus = true
59 }
60 ....
61
62 // 4. Connect with the TempleAction event stream
63 lifecycleScope.launch {

```

```

64     repeatOnLifecycle(Lifecycle.State.RESUMED) {
65         templeActionViewModel.state.collect {
66             FLogger.i("DemoActivity", "action = $it")
67             when (it) {
68                 is TempleAction.DoubleClick -> {
69                     // Handle double-click exit logic
70                     finish()
71                 }
72                 else -> fixPosFocusTracker?.handleFocusTargetEvent(it)
73             }
74         }
75     }
76 }

```

This process handles focus management for views at the same level. `FixPosFocusTracker` can itself be used as a target for `FocusInfo`, allowing nested structures to handle complex focus switching scenarios.

For `RecyclerView`, focus management differs depending on whether focus positions are fixed:

- **Fixed Focus Positions:** Use `RecyclerViewSlidingTracker`.
- **Moving Focus Positions:** Use `RecyclerViewFocusTracker`.

Refer to `FixedFocusPosRVActivity` and `MovedFocusPosRVActivity` in the SDK demo for implementation details.

For other scenarios, such as the overall focus switching in a custom View, you can handle it uniformly by implementing the `IFocusable` interface. Only when the `hasFocus` property is `true` will it respond to events.

Both `RecyclerViewSlidingTracker`, `RecyclerViewFocusTracker`, and `IFocusable` objects can be used as targets for `FocusInfo`.

3. Touch Events & Event Response

Currently, both left and right temple arms can trigger MotionEvent TP events. Unlike the 2D TP events on mobile phones, TP events generated by the temple arms are 1D, with the X coordinate

changing dynamically and the Y coordinate fixed. In addition to the temple arms, ring accessories (after pairing) also generate TP events.

Event Response Mechanism

When TP events are dispatched to the Activity or View level, the raw MotionEvent is passed to the `TouchDispatcher` during the dispatch process. The `TouchDispatcher` converts native TP events into gestures such as single tap, long press, double tap, triple tap, forward swipe, and backward swipe (some gestures have default system sounds), which are exposed via the `CommonTouchCallback` interface. Callers only need to implement business logic based on these gestures.

Based on `TouchDispatcher` and `CommonTouchCallback`, the following Activity-level encapsulations are provided:

- `BaseTouchActivity`: Automatically registers gesture listeners.
- `BaseEventActivity`: Inherits from `BaseTouchActivity` and converts gestures into Kotlin Flow event streams (`BaseMirrorActivity` inherits from `BaseEventActivity`), mapping events to `TempleAction` subclasses.

Event Listening Example:

Code block

```
1  class TPEventActivity : BaseMirrorActivity<ActivityTpEventBinding>() {
2      private var fixPosFocusTracker: FixPosFocusTracker? = null
3      override fun onCreate(savedInstanceState: Bundle?) {
4          super.onCreate(savedInstanceState)
5          initFocusTarget()
6          initEvent()
7      }
8
9      private fun initFocusTarget() {
10         val focusHolder = FocusHolder(false)
11         mBindingPair.setLeft {
12             focusHolder.addFocusTarget(
13                 FocusInfo(
14                     btnEvent,
15                     eventHandler = { action -> handleAction(action) },
16                     focusChangeHandler = { hasFocus ->
17                         mBindingPair.updateView {
18                             triggerFocus(hasFocus, btnEvent,
19                             mBindingPair.checkIsLeft(this))
20                         }
21                     }
22             )
23         }
```

```

20         }
21     )
22 )
23     focusHolder.currentFocus(mBindingPair.left.btnEvent)
24 }
25     fixPosFocusTracker = FixPosFocusTracker(focusHolder).apply {
26         focusObj.reqFocus()
27     }
28 }
29
30 private fun handleAction(action: TempleAction) {
31     when (action) {
32         is TempleAction.LongClick -> {
33             FToast.show("LongClick")
34         }
35
36         is TempleAction.Click -> {
37             FToast.show("Click")
38         }
39
40         is TempleAction.DoubleClick -> {
41             FToast.show("DoubleClick")
42             finish()
43         }
44
45         is TempleAction.TripleClick -> {
46             FToast.show("TripleClick")
47         }
48
49         is TempleAction.SlideBackward -> {
50             FToast.show("SlideBackward")
51         }
52
53         is TempleAction.SlideForward -> {
54             FToast.show("SlideForward")
55         }
56
57         is TempleAction.SlideUpwards -> {
58             FToast.show("SlideUpwards")
59         }
60
61         is TempleAction.SlideDownwards -> {
62             FToast.show("SlideDownwards")
63         }
64
65         else -> Unit

```



```

18         showDialog()
19     }
20     else -> Unit
21 }
22 }
23 }
24 }
25 }

```

For apps migrated from X2 SDK to X3:

- Use `DeviceUtil.isX3Device()` to detect Rayneo X3:

Code block

```

1  if (DeviceUtil.isX3Device()) {
2      // Run on Rayneo X3
3  } else {
4      // Run on Rayneo X2
5  }

```

3.1 Changes to `CommonTouchBack` in Rayneo X3 SDK

New methods (not available in X2 series):

Code block

```

1  // Upward swipe
2  open fun onTPSlideUpwards(args: FlingArgs): Boolean {
3      return false
4  }
5
6  // Downward swipe
7  open fun onTPSlideDownwards(args: FlingArgs): Boolean {
8      return false
9  }
10
11 // Double-finger tap
12 open fun onTPDoubleFingerClick() {
13 }
14
15 // Double-finger long press
16 open fun onTPDoubleFingerLongClick() {

```

```
17 }
```

Modified method:

Code block

```
1 onTPSlideContinuous(delta: Float, longClick: Boolean = false, vertical: Boolean  
    = false)
```

- Added `vertical` parameter to indicate vertical swipe direction.

New feature:

- `filterMode` property in `CommonTouchCallback`:
 - `OnlyX`: `onTPSlideContinuous` returns only X-axis swipe data.
 - `OnlyY`: `onTPSlideContinuous` returns only Y-axis swipe data.
 - Supported only on Rayneo X3.
- `TemplateAction.TpSlideContinuous` now includes swipe direction (X/Y axis).

3.2 `SlideBackward` & `SlideForward` Handling

The mapping between touchpad swipes and UI events on Rayneo glasses has two modes (configurable in settings, see video for path):



Natural Mode (Default)

- **Touchpad-UI mapping:**

- Swipe from temple to lens: `TempleAction.SlideBackward`
- Swipe from lens to temple: `TempleAction.SlideForward`



Non-Natural Mode

- **Touchpad-UI mapping:**

- Swipe from temple to lens: `TempleAction.SlideForward`
- Swipe from lens to temple: `TempleAction.SlideBackward`



4. Following Hand Effect (Smooth Scrolling)

While the `TouchDispatcher` and `CommonTouchCallback` suffice for general scenarios, achieving the smooth scrolling effect seen in mobile lists is challenging due to two main limitations:

- Event Processing Layer:** `TouchDispatcher` processes raw events into gestures, but smooth scrolling requires direct handling of raw events.
- Coordinate System Mismatch:** Raw TP events have a fixed Y-coordinate, while lists can be horizontal or vertical and only respond to events within their bounds. This requires dynamically adjusting coordinates:
 - Horizontal Lists:** Modify the Y-coordinate based on the list's center.
 - Vertical Lists:** Swap X and Y coordinates and fix the X-coordinate.

Implementation Approach:

- Continue feeding raw events to `TouchDispatcher` for gesture detection (clicks, taps, etc.), but ignore forward/backward swipes.
- Use a fixed focus position on the `RecyclerView` to intercept the raw event stream.
- Generate new `MotionEvent` instances with adjusted coordinates, then feed them to the `RecyclerView` and recycle them afterward.
- The new event stream handles only scrolling, not click events.

Three Common Scenarios:

- Fixed Focus + List Scrolling:** Use `RecyclerViewSlidingTracker`.
- Moving Focus + List Scrolling:** Use `RecyclerViewFocusTracker` with distance-based focus changes.

3. Moving Focus + Fixed View: Use `FixPosFocusTracker`.

Key Logic for Moving Focus:

After sliding a threshold distance (e.g., 50dp), trigger focus changes (forward/backward or up/down) based on the swipe direction.

Example Code for Fixed Focus (Coordinate Adjustment):

Code block

```
1  // Intercept raw events for smooth scrolling
2  recyclerViewSlidingTracker.observeOriginMotionEventStream(
3      motionEventDispatcher
4  ) { event ->
5      // Create a new MotionEvent with adjusted coordinates (swap X and Y as
        needed)
6      MotionEvent.obtain(
7          event.downTime,
8          event.eventTime,
9          event.action,
10         320f, // Fixed X-coordinate (adjust based on layout)
11         event.x, // Use original X as new Y for vertical lists
12         event.metaState
13     ).apply {
14         val e = this
15         FLogger.d(
16             "onReceiveEvent: x = ${e.x}, y = ${e.y}, action =
                ${e.actionName()}, deviceId = ${e.deviceId}"
17         )
18     }
19 }
20
21 lifecycleScope.launchWhenResumed {
22     val templeActionViewModel =
23         ViewModelProvider(this@FixedFocusPosRVActivity).get<TempleActionViewModel>()
24     templeActionViewModel.state.collect {
25         if (!favoriteTracker.focusObj.hasFocus || !this.isActive ||
            it.consumed) {
26             return@collect
27         }
28         recyclerViewSlidingTracker.handleActionEvent(it) { action ->
29             when (action) {
30                 is TempleAction.DoubleClick -> finish()
```

```

31         is TemplateAction.Click -> {
32             if (!action.consumed) {
33                 (mBindingPair.left.recyclerView.adapter as
FixedFocusPosAdapter)
34                     .getCurrentData()?.apply {
35                         FToast.show(displayName)
36                     }
37             }
38         }
39         else -> {}
40     }
41 }
42 }
43 }

```

Explanation:

- `motionEventDispatcher` (from `BaseTouchActivity`) intercepts the raw event
- `observeOriginMotionEventStream` adjusts coordinates (swaps X/Y for vertical lists) to make the events consumable by the `RecyclerView`.
- See `FixedFocusPosRVActivity` for the complete implementation.

5. 3D Effect Implementation

In the Android native environment, a pseudo-3D effect is achieved using binocular parallax. The core principle involves offsetting views in the left and right layouts in opposite directions to create depth perception:

- **Left Layout:** Views are shifted right.
- **Right Layout:** Views are shifted left.
- Varying offset values create different depths, resulting in a 3D effect when viewed through both eyes.

SDK Methods:

Code block

```

1 // Apply 3D effect to both left and right views simultaneously

```

```

2  make3DEffect(left.slideView, right.slideView, true, 10f)
3
4  // Apply 3D effect to a single view (specify left/right)
5  make3DEffectForSide(this.ivSelectHover, isLeft, true)

```

Parameters:

- `leftView/rightView` : Views from left/right layouts.
- `isLeft` : Boolean indicating if the view is from the left layout.
- `enabled` : Enable/disable the effect.
- `depth` : Depth intensity (higher values create stronger 3D effects).

6. Audio Capture Modes

Developing audio applications for RayNeo glasses follows similar steps to general Android audio development. For microphone recording, RayNeo glasses offer specialized recording modes, which differ slightly between X2 and X3:

X2 Supported Recording Modes

Each APK must set the recording mode via

`AudioManager.setParameter("audio_source=xxxx")` when using the microphone:

Mode	Description	Example
<code>audio record</code>	Not yet configured	<code>setParameters("audio_source_record=sound")</code>
<code>camera record</code>	Uses 2 temple microphones, captures full sound without noise cancellation	<code>setParameters("audio_source_record=camcorder")</code>
<code>translation</code>	Uses 3 microphones, ignores wearer's voice, captures external sound	<code>setParameters("audio_source_record=translation")</code>
<code>voice assistant</code>	Uses 2 temple microphones, prioritizes wearer's voice	<code>setParameters("audio_source_record=voiceassistant")</code>

Best Practice: To ensure exclusive microphone access, reset the mode to off when finishing recording:

```
code>setParameters("audio_source_record=off")
```

X3 Supported Recording Modes

Mode	Precondition (AudioRecord)	setParameters	Description
OFF	Exit Recording	<code>audioManager.setParameters("audio_source_record=off")</code>	Remember to release the microphone when the APK exits, and settings required when exiting the recording.
RECORD_TRANSLATION	<code>device.id == SPEAKER_MIC ;CHANNEL_IN_MONO;</code> The sampling rate of the sending translation engine is generally set to 16k;	ExampleRecordTranslation.java	Before calling, the microphone records surrounding voices, suitable for 10 minutes/on-site translation
CAMCORDER	<code>device.id == SPEAKER_MIC; CHANNEL_IN_STEREO;</code> The sampling rate is set as needed, with the maximum not exceeding 48kHz;	ExampleRecordCamcorder.java	Left and right temple microphone recording, omnidirectional sound, suitable for video recording scenarios
VOICE_RECOGNITION	<code>device.id == SPEAKER_MIC; CHANNEL_IN_MONO;</code> The recognition engine generally requires a sample rate of 16kHz and single-channel;	ExampleRecordRecognition.java	Three microphones are used to capture only the wearer's voice, which is sent to the recognition engine for recognition without considering audio quality, suitable for scenarios such as voice assistants and navigation
VOICE_COMMUNICATION	<code>device.id == SPEAKER_MIC</code>	ExampleRecordCommunication.java	Calls the front microphone and temple microphone, capturing only the voice, suitable for online calls, online meetings, etc.

7. Camera Development

Camera development for RayNeo glasses is identical to standard Android camera development. Refer to developer.android.com for details and ensure proper permission handling and resource release. The `camera2` API is recommended, with a detailed implementation in the sample `CameraActivity` demonstrating:

- Dual-screen camera preview
- Capturing single frames from the camera

Camera Resolution Enumeration Example:

```

1 private fun enumerateCameraResolutions() {
2     val cameraManager = getSystemService(CAMERA_SERVICE) as CameraManager
3     val cameraIdList = cameraManager.cameraIdList
4
5     for (cameraId in cameraIdList) {
6         val characteristics = cameraManager.getCameraCharacteristics(cameraId)
7         val map =
            characteristics.get(CameraCharacteristics.SCALER_STREAM_CONFIGURATION_MAP)
8
9         if (map != null) {
10             val previewSizes = map.getOutputSizes(SurfaceTexture::class.java)
11             val pictureSizes = map.getOutputSizes(ImageFormat.JPEG)
12
13             FLogger.d("Camera ID: $cameraId")
14             FLogger.d("Supported Preview Sizes:")
15             for (size in previewSizes) {
16                 FLogger.d("  ${size.width}x${size.height}")
17             }
18
19             FLogger.d("Supported Picture Sizes:")
20             for (size in pictureSizes) {
21                 FLogger.d("  ${size.width}x${size.height}")
22             }
23         }
24     }
25 }

```

Source code example of SnapdragonCamera (Image capture works normally when the AE convergence status = 2):

<https://github.com/Lh1600852534/snapdragonCamera/blob/master/src/com/android/camera/CaptureModule.java>

VGA camera

There is also a standalone VGA camera on the Rayneo X3 glasses, which is specifically used for spatial positioning. The supported parameters are as follows:

Code block

```

1  2025-09-25 18:13:32.490 1583-1583 camera
   com.ffalcon.mercury.android.sdk      D Camera ID: 1
2  2025-09-25 18:13:32.490 1583-1583 camera
   com.ffalcon.mercury.android.sdk      D Supported Preview Sizes:

```

3	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	640x480
4	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	640x400
5	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	640x360
6	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	480x360
7	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	352x288
8	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	320x240
9	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	320x180
10	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	176x144
11	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	Supported Picture Sizes:
12	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	640x480
13	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	640x400
14	2025-09-25 18:13:32.491 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	640x360
15	2025-09-25 18:13:32.492 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	480x360
16	2025-09-25 18:13:32.492 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	352x288
17	2025-09-25 18:13:32.492 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	320x240
18	2025-09-25 18:13:32.492 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	320x180
19	2025-09-25 18:13:32.492 1583-1583	camera	
	com.ffalcon.mercury.android.sdk	D	176x144

8. IMU Data Acquisition

Accelerometer, gyroscope, and magnetometer data can be obtained and displayed via Android's native API. For details, please refer to the `IMUActivity` in the demo.

For applications that require device posture (e.g., rotation angle), using raw sensor data directly is extremely complex. Android provides **software sensors**, which automatically fuse data from multiple hardware sensors to deliver smoother and more accurate results.

Game Rotation Vector Sensor

This is the most suitable sensor for games and real-time posture tracking. It does not use a magnetometer, so it is not affected by magnetic field interference. However, it cannot provide absolute orientation (e.g., north-pointing).

Code block

```
1  // Obtain the sensor in onCreate
2  private var gameRotationVectorSensor: Sensor? = null
3
4  // Add inside the onCreate method:
5  gameRotationVectorSensor =
    sensorManager.getDefaultSensor(Sensor.TYPE_GAME_ROTATION_VECTOR)
6
7  // Register in onResume
8  gameRotationVectorSensor?.let {
9      sensorManager.registerListener(this, it,
        SensorManager.SENSOR_DELAY_FASTEST)
10 }
11
12 // Process data in onSensorEvent
13 Sensor.TYPE_GAME_ROTATION_VECTOR -> {
14     // event.values contains 4 components: [xsin(θ/2), ysin(θ/2), zsin(θ/2),
        cos(θ/2)]
15     // This is a Quaternion representing rotation
16     val x = event.values[0]
17     val y = event.values[1]
18     val z = event.values[2]
19     val w = event.values[3]
20     // You can convert the quaternion to Euler angles (pitch, roll, yaw) for
        easier understanding
21     // Note: Since the magnetometer is not used, the yaw angle is relative and
        will drift.
```

Convert quaternion to Euler angles (simple example)

Code block

```
1  private fun quaternionToEuler(x: Float, y: Float, z: Float, w: Float):
    FloatArray {
2      val euler = FloatArray(3)
3
4      // Pitch angle (pitch, rotation around X-axis)
5      val sinP = 2.0f * (w * x + y * z)
6      val cosP = 1.0f - 2.0f * (x * x + y * y)
7      euler[0] = Math.atan2(sinP.toDouble(), cosP.toDouble()).toFloat()
```

```

8
9    // Roll angle (roll, rotation around Y-axis)
10   val sinR = 2.0f * (w * y - z * x)
11   euler[1] = if (Math.abs(sinR) >= 1) {
12       (Math.PI / 2).toFloat() * Math.signum(sinR)
13   } else {
14       Math.asin(sinR.toDouble()).toFloat()
15   }
16
17   // Yaw angle (yaw, rotation around Z-axis)
18   val sinY = 2.0f * (w * z + x * y)
19   val cosY = 1.0f - 2.0f * (y * y + z * z)
20   euler[2] = Math.atan2(sinY.toDouble(), cosY.toDouble()).toFloat()
21
22   // Convert radians to degrees (optional)
23   euler[0] = Math.toDegrees(euler[0].toDouble()).toFloat()
24   euler[1] = Math.toDegrees(euler[1].toDouble()).toFloat()
25   euler[2] = Math.toDegrees(euler[2].toDouble()).toFloat()
26
27   return euler
28 }

```

Sampling Rate Constant

When `registering a listener`, you can choose different sampling rates:

- `SENSOR_DELAY_FASTEST` : Fastest, with the highest possible frequency.
- `SENSOR_DELAY_GAME` : Suitable for game rate.
- `SENSOR_DELAY_NORMAL` : Default rate, suitable for screen orientation changes.
- `SENSOR_DELAY_UI` : Suitable for User Interface changes.

Best Practice: Always remember to call `sensorManager.unregisterListener(this)` in `onPause()` to avoid background power consumption.

9. Mobile Connection & GPS Streaming

Mobile Connection Status

Monitor phone connection status using the `MobileState` class:

Code block

```

1   private fun collectBleStatus() {
2       MobileState.isMobileConnected().onEach { isConnected ->

```

```

3         FLogger.d("isMobileConnected:$isConnected")
4         mBindingPair.updateView {
5             tvBleStatus.text = if (isConnected) "connect" else "disconnect"
6         }
7     }.launchIn(lifecycleScope)
8 }

```

GPS Streaming from Connected Phone

1. Integrate the IPC SDK (refer to [IPC SDK for Android](#))
2. Implement GPS streaming as shown in `IPC SDK Sample` or `APIActivity` in the main sample.

Note: The `onResponse` callback runs on a background thread; ensure UI updates are performed on the main thread.

Code block

```

1  private val response = OnResponseListener { response ->
2      if (response?.getData() == null) return@OnResponseListener
3      try {
4          val jo = JSONObject(response.getData())
5          if (jo.has("mLatitude") && jo.has("mLongitude") &&
jo.has("mAltitude")) { // GPS data
6              val mProvider = jo.getString("mProvider")
7              val mTime = jo.getLong("mTime")
8              val mElapsedRealtimeNanos = jo.getLong("mElapsedRealtimeNanos")
9              val mLatitude = jo.getDouble("mLatitude")
10             val mLongitude = jo.getDouble("mLongitude")
11
12             // Switch to UI thread for updates
13             runOnUiThread {
14                 mBindingPair.updateView {
15                     tvTvLocationInfo.text = "$mLatitude,$mLongitude"
16                 }
17             }
18         }
19     } catch (e: JSONException) {
20         e.printStackTrace()
21     }
22 }

```

